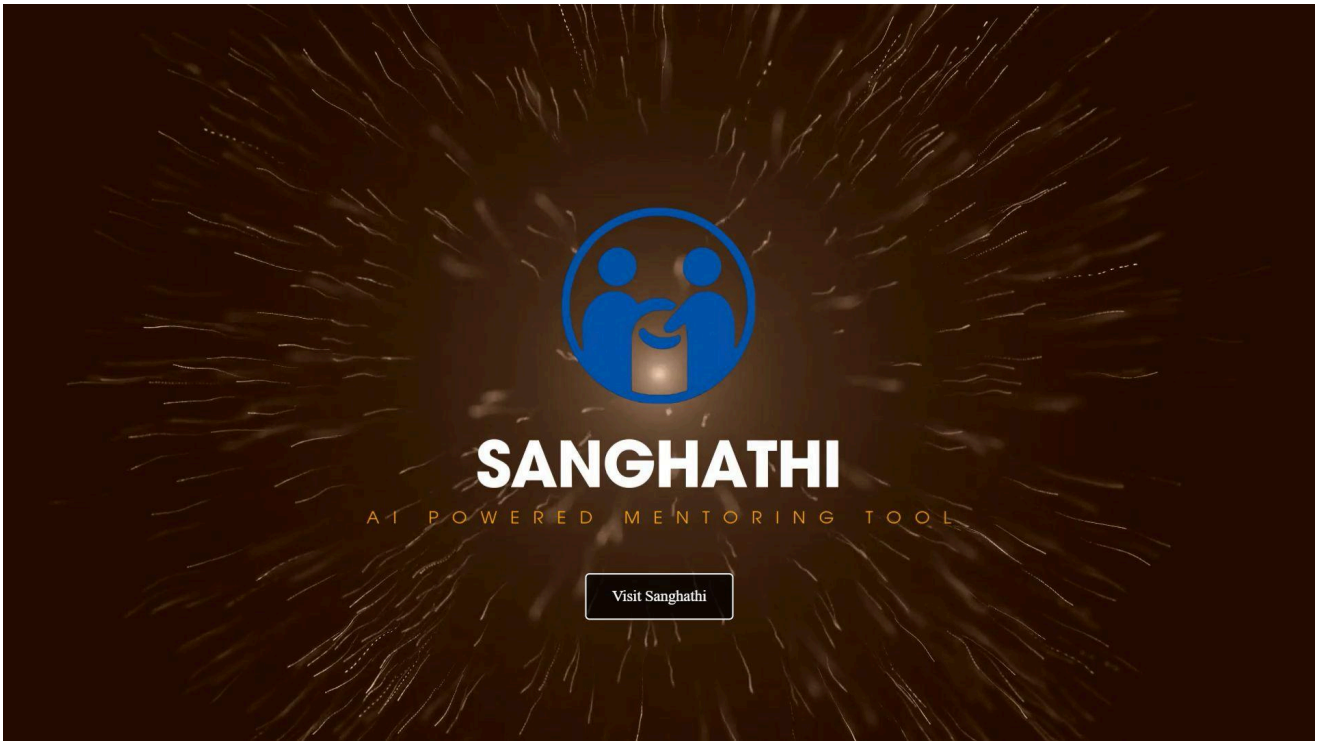
# SANGHATHI

## AI-Powered Mentoring Platform




## Master Project Report

Prepared for Mentor Review & Project Documentation Handoff

| | |
|---|---|
| **Date** | April 19, 2026 |
| **Platform** | sanghathi.com |
| **Frontend Repo** | github.com/Sanghathi/sanghathi-Frontend |
| **Backend Repo** | github.com/Sanghathi/sanghathi-Backend |
| **License** | MIT |

# 1. Executive Overview

Sanghathi is a full-stack, AI-powered mentoring platform built to unify and digitize academic mentoring workflows across an institution. It brings together students, faculty, HODs, directors, and administrators into a single role-aware web application that handles student lifecycle data, mentoring sessions, real-time communication, academic performance tracking, career guidance, and AI-assisted insights — all under one roof.

The platform is production-deployed, actively used, and built with a modern tech stack including React + Vite on the frontend, Node.js + Express on the backend, and MongoDB Atlas as the primary database — with cloud infrastructure spanning Netlify, Fly.io, Cloudflare, and Resend.

## 1.1 Core Value Proposition

- Centralizes student mentoring data across academic, career, placement, and performance domains
- Role-based access for Students, Faculty, HOD, Director, and Admin with strict authorization
- Real-time communication via Socket.IO — private threads, group conversations, offline logs
- AI-powered Campus Buddy for context-aware student guidance
- End-to-end deployment pipeline with SEO, analytics, and observability tooling

# 2. Technology Stack

## 2.1 Frontend

| Technology | Details |
|---|---|
| **Framework** | React 18 + Vite (SPA, optimized build pipeline) |
| **Language** | JavaScript (ES2022+) |
| **UI Library** | Material UI (MUI) + Emotion for styled components |
| **Routing** | React Router DOM with role-based protected routes |
| **Forms** | React Hook Form + Yup schema validation |
| **Real-time** | Socket.IO Client for live messaging and notifications |
| **Animation** | Framer Motion for smooth UI transitions |
| **Calendar** | FullCalendar for meeting scheduling UI |
| **AI SDK** | Google Generative AI SDK (Campus Buddy) |
| **SEO** | React Helmet Async — dynamic metadata per page |

| | |
|---|---|
| **Testing** | Vitest + Testing Library |
| **Package Mgr** | Bun (fast builds and installs) |
| **Deployment** | Netlify with SPA rewrite rules |

## 2.2 Backend

| Technology | Details |
|---|---|
| **Runtime** | Node.js + Bun-compatible packaging |
| **Framework** | Express.js with domain-first API structure |
| **Database** | MongoDB + Mongoose ODM |
| **Authentication** | JWT-based auth with refresh token support |
| **Real-time** | Socket.IO for bidirectional messaging |
| **Email** | Resend (transactional mail via send.sanghathi.com) |
| **Media Storage** | Cloudinary for file and image uploads |
| **Security** | Helmet, rate limiting, mongo-sanitize, xss-clean |
| **Logging** | Winston logging stack (structured, leveled) |
| **Validation** | Joi/Zod schema validation |
| **API Docs** | Swagger (OpenAPI) auto-generated docs |
| **Containerization** | Docker image (oven/bun:1.3.10-slim base) |
| **Deployment** | Fly.io machine-based runtime |

# 3. Functional Architecture

Sanghathi is structured around six major product domains, each mapped to distinct backend API namespaces and corresponding frontend module routes. Every domain is protected with role-based access control enforced at both the API middleware and frontend routing layers.

## 3.1 Product Domains

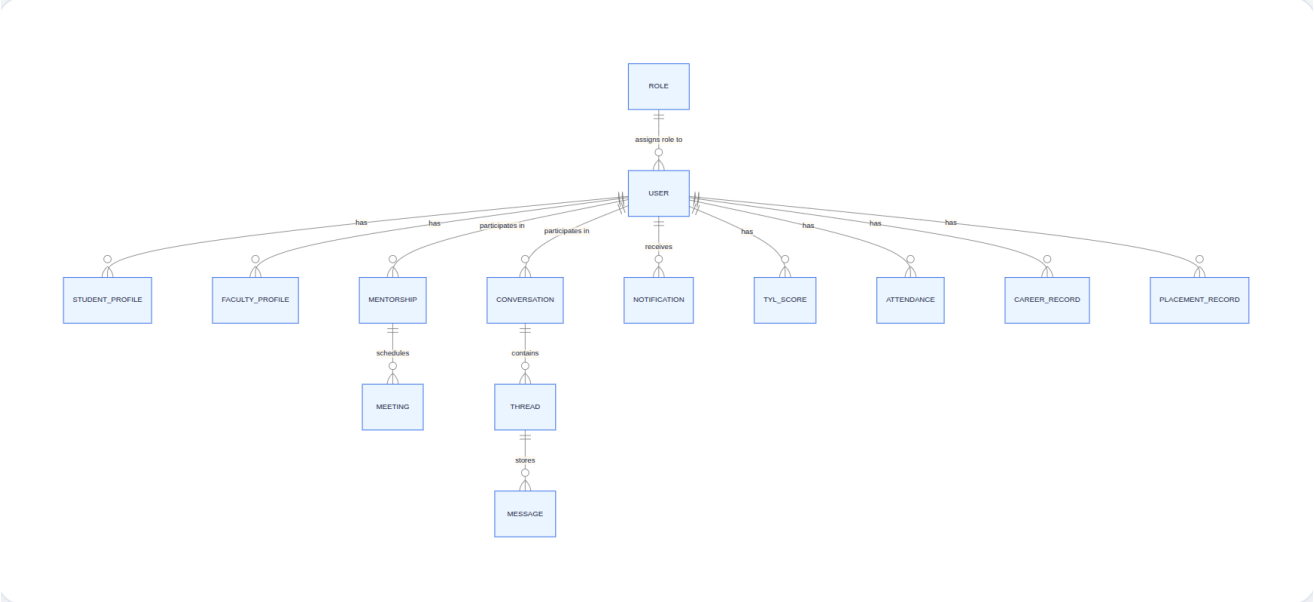
| # | Domain | Description |
|---|---|---|
| 1 | **Auth & Role-Based Access** | Login, signup, password reset, JWT-protected routes per role (Student / Faculty / HOD / Director / Admin) |
| 2 | **Student Lifecycle** | Admissions, academics, attendance, profile data, guardian/contact info |

| | | |
|---|---|---|
| 3 | **Mentorship Operations** | Mentor-mentee mapping, dashboards, offline session logs, meeting scheduling |
| 4 | **Career & Placement** | Career counseling, MOOC, mini-projects, internships, club activities, placement records |
| 5 | **Communication** | Threads, private & group conversations, real-time messaging via Socket.IO |
| 6 | **Academic Performance** | IAT scores, external marks, PO attainment, TYL score tracking |
| 7 | **Notifications** | Alerting flows, meeting reminders, dashboard-level updates |
| 8 | **AI Features** | Campus Buddy — AI-assisted student guidance via Gemini API |

## 3.2 Role-Based Navigation

Every dashboard and navigation tree is driven by the user's role. Below are the five primary portal types:

| Student | Faculty | HOD | Director | Admin |
|---|---|---|---|---|
| Profile & Academics | Mentee List | Department Oversight | Institution Analytics | User & System Admin |

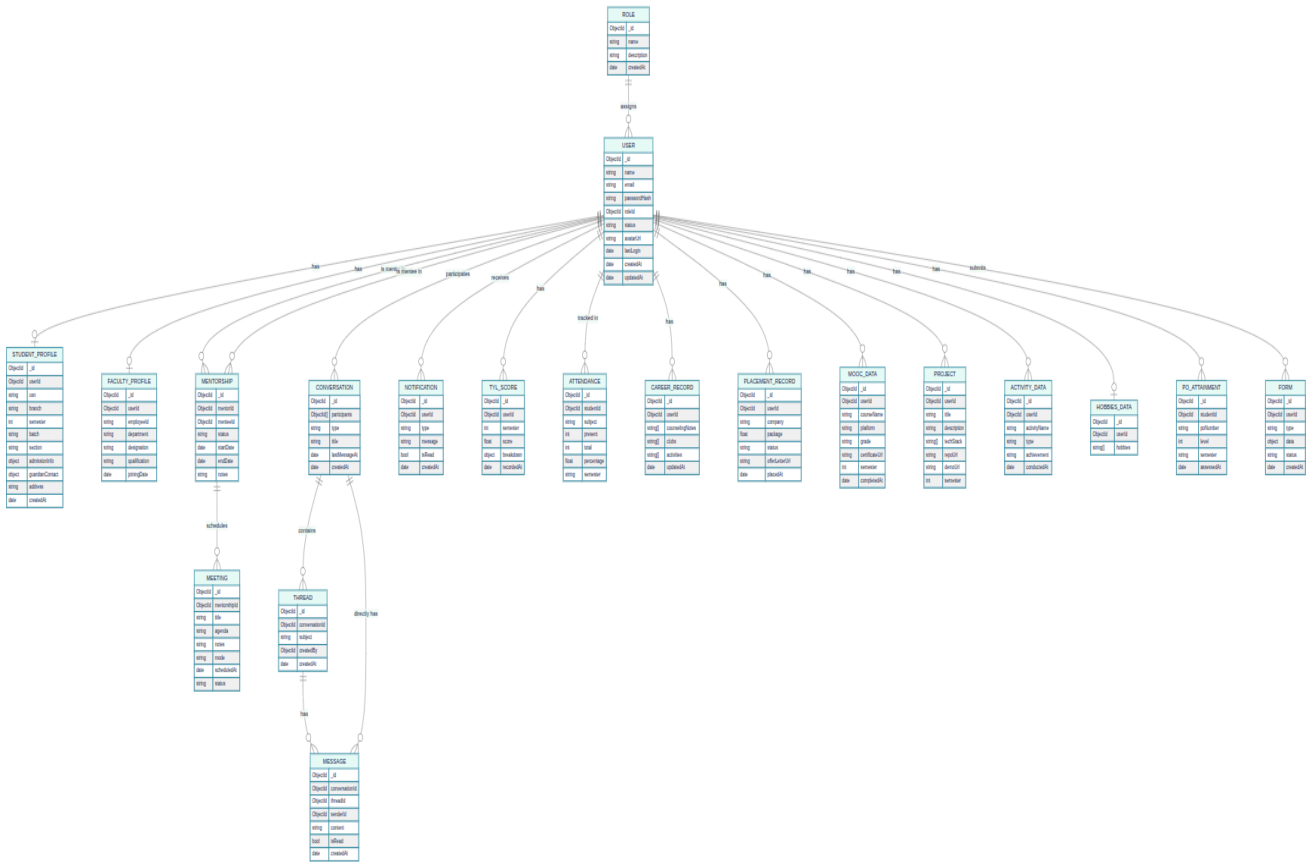



## 3.3 Backend API Surface

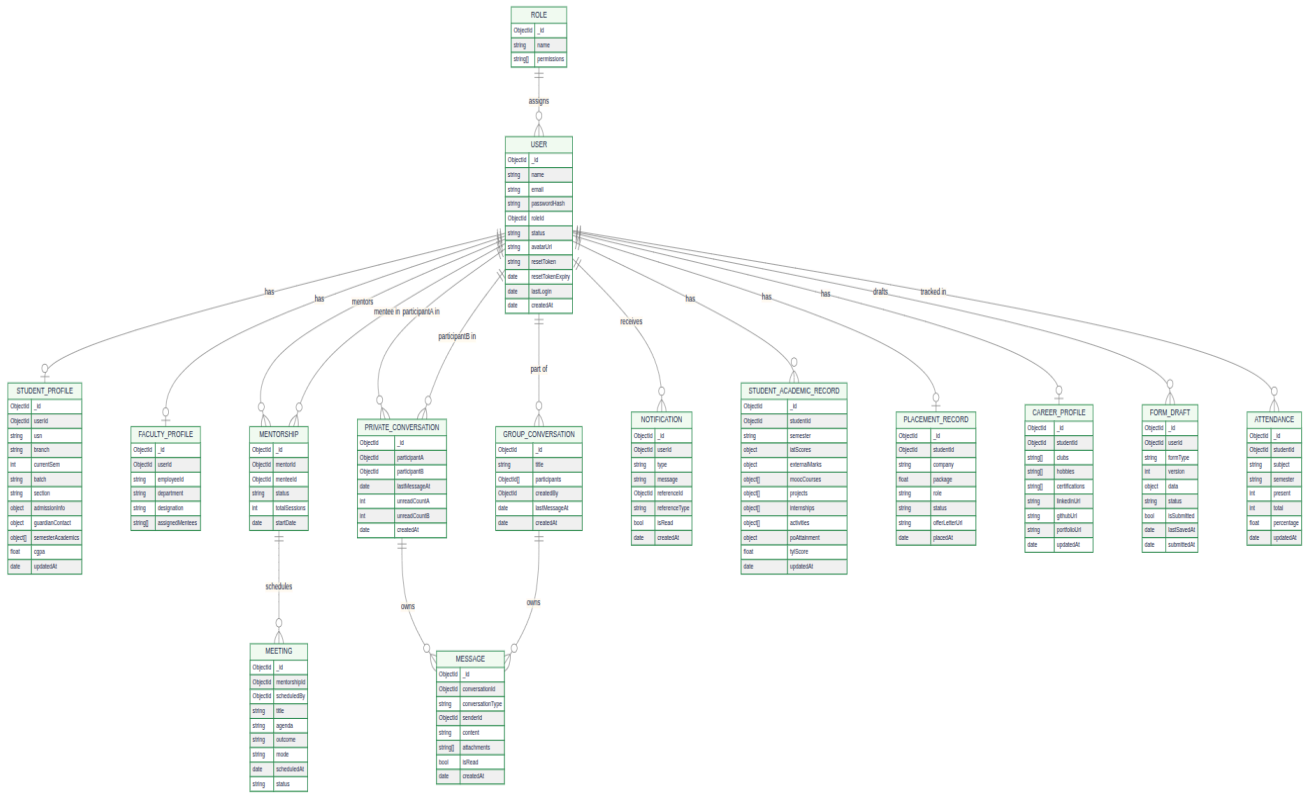
The backend exposes a domain-first REST API structure. Major API groups:

| | | |
|---|---|---|
| `/api/users`<br>`/api/students`<br>`/api/faculty` | `/api/messages`<br>`/api/conversations`<br>`/api/threads` | `/api/meetings`<br>`/api/notifications`<br>`/api/campus-buddy` |
| `/api/mentors`<br>`/api/mentorship`<br>`/api/placement` | `/api/career-counselling`<br>`/api/mooc-data`<br>`/api/project` | `/api/tyl-scores`<br>`/api/po-attainment`<br>`/api/admin` |

## 3.4 DataBase- Current Needs Fixes



## 3.5 DataBase- Planned




# 4. Deployment & Infrastructure

Sanghathi is deployed across a modern cloud stack with separation between frontend hosting, backend runtime, database, email, and DNS. All services are production-grade and actively running.

| Service Area | Platform / Service | Notes |
|---|---|---|
| **Frontend Hosting** | **Netlify** | Bun + Vite build; SPA rewrite; security headers; dist directory published |
| **Backend Runtime** | **Fly.io + Docker** | Dockerized Bun/Node service; PORT env-configurable; machines-based scaling |
| **Database** | **MongoDB Atlas** | cmrit database; collections, monitoring, vector search enabled |
| **DNS & Edge** | **Cloudflare** | DNS, caching, web analytics, Core Web Vitals, DDoS protection |
| **Email** | **Resend** | Verified sending domain: send.sanghathi.com; password reset + notifications |
| **Media Storage** | **Cloudinary** | Profile photos, file uploads, image transformation |

| | | |
|---|---|---|
| **Real-time** | **Socket.IO** | Bidirectional comms for messaging, presence, notifications |
| **Search & SEO** | **Google Search Console** | Property configured for sanghathi.com; sitemap submitted |
| **AI Engine** | **Google Gemini API** | Campus Buddy AI guidance and generative responses |

## 4.1 Deployment Pipeline

The deployment flow follows a clean CI/CD-adjacent pipeline from developer push to end user:

- Developer pushes code to GitHub (Sanghathi org repos)
- Frontend: Netlify auto-detects push → runs 'bun run build' → deploys dist/ → SPA rewrite active
- Backend: Docker image built from oven/bun:1.3.10-slim → deployed to Fly.io runtime machines
- Cloudflare handles DNS routing for both frontend and backend, adding edge caching and analytics
- MongoDB Atlas serves as the persistent data layer; Resend handles all transactional email delivery
- Google Search Console tracks indexing coverage post sitemap submission

# 5. Contributors

Sanghathi is a collaborative engineering project with contributions spread across frontend, backend, infrastructure, and documentation. Below is a summary of all team members and their primary contribution domains.

| Contributor | GitHub Handle | Primary Contribution Areas |
|---|---|---|
| **Kethan VR** | Kethanvr | Security, role routing, SEO, migrations, deployment hardening, docs |
| Shovan mondal | shovan-mondal | Core architecture, development coordination, product direction |
| Monu kumar | monu564100 | Feature modules, debugging support, cross-module improvements |
| | | |
| Kulsum | Kulsum06 | UI and workflow improvements, module-level development |
| Sai-Eman | Sai-Emani25 | Feature implementation, module support, issue fixes |
| advitha | advitha24 | Frontend and feature support contributions |
| | | |

# 6. Kethan VR — Detailed Contribution Report

This section provides a comprehensive, evidence-backed breakdown of Kethan VR's technical contributions to the Sanghathi project. All metrics are derived from git history filtered by author "Kethan" since April 13, 2026, covering both the frontend and backend repositories.

## 6.1 Quantitative Metrics at a Glance

| | | | |
|---|---|---|---|
| **88** Total Commits | **23,134** Lines Added | **5,423** Lines Deleted | **17,711** Net Lines Written |
| **607** File Change Events | **334** Unique Files Touched | **272** Code/Config Files | **7 Days** Active Sprint |

## 6.2 Repository Breakdown

| Metric | Frontend Repository | Backend Repository |
|---|---|---|
| **Commits** | 58 | 30 |
| **Lines Added** | 13,940 | 9,194 |
| **Lines Deleted** | 4,198 | 1,225 |
| **Net Line Change** | 9,742 | 7,969 |
| **File Change Events** | 404 | 203 |
| **All Unique Files** | 201 | 133 |
| **Code/Config Files Only** | 141 | 131 |

## 6.3 Contribution Areas — Detailed Breakdown

### A. Security & Access Control

Problem: Several API routes were unprotected or had inconsistent role gating — a serious production risk given that the platform handles sensitive student data across multiple privileged roles.

- Added and verified route protection middleware across all role paths (Student, Faculty, HOD, Director, Admin)

- Expanded role validation tests to cover edge cases and unauthorized access attempts
- Normalized role-based navigation behavior to prevent cross-role route leakage
- Implemented integration tests covering auth guards and protected-route 403 scenarios

### B. Reliability & Logging

Problem: Inconsistent console.log usage scattered across both repos made debugging a nightmare — no structured logging, no severity levels, no observable patterns.

- Standardized Winston-based logger usage across all backend code paths
- Removed scattered, inconsistent console.log usage from both frontend and backend
- Improved observability so production issues can be diagnosed from structured log output

### C. Test Coverage & Regression Safety

Problem: Gaps in auth and route authorization testing meant that regressions in security-critical flows could slip into production undetected.

- Added progressive test phases: Phase 1 (auth guards), Phase 2 (logging), Phase 3 (TODO/FIXME cleanup), Phase 4 (test coverage expansion)
- Phase 5–6: Protected route integration tests and role-based 403 integration tests
- Phase 7: Unified testing and server start scripts for repeatable test execution

### D. Data Safety & Backup Operations

Problem: Backup workflow inconsistencies and drift between local and Atlas backup processes created data loss risk.

- Authored and documented Cloudflare + Atlas backup runbooks
- Migrated operational commands to Bun runtime for consistency
- Standardized root-level scripts for backup execution and rollback procedures
- Implemented Atlas-first backup strategy with clear rollback paths

### E. Messaging Schema Migration

Problem: Conversation/message model had structural constraints requiring a phased migration to parent-owned fields and safer indexing — a high-risk operation on live data.

- Designed and executed phased migration scripts with feature-flag rollout strategy
- Implemented data backfills and unique-index enforcement for one-to-one conversations
- Executed IAT-6 CSV ingest with verification scripts for academic data integrity
- Built local IAT ingest tools and rollback scripts for safe data operations

### F. Password Reset & Email Delivery

Problem: Password reset flow lacked production-quality email templates and had deliverability issues that impacted user trust.

- Migrated password reset emails to Resend with verified sending domain (send.sanghathi.com)

- Designed and implemented standardized HTML email templates for reset and notification flows
- Hardened reset route coverage with integration tests
- Improved template safety, mobile responsiveness, and deliverability scores

## G. Frontend UX & Responsiveness

Problem: Multiple dashboard pages, forms, and data views lacked mobile consistency — a critical issue for a student-facing platform used on phones.

- Phase 3 responsive hardening across admin, faculty, and student dashboard flows
- Rolled out shared hero/loading components for consistent skeleton loading behavior
- Login page: full mobile responsive redesign including illustration parity across breakpoints
- TYL scorecard, profile wrapper, companion views — all unified to shared mobile shell
- Admin upload history restore workflow and data visibility improvements

## H. Developer Pages, Footer & SEO

Problem: The developer showcase section and footer were incomplete, and the site lacked basic SEO infrastructure that would affect discoverability and search ranking.

- Built full developer profile routing — individual cards, detail views, about-developers page
- Added shortcards with Read More expansion pattern on the developer page
- Reworked footer structure with inline contact and layout consistency
- Added sitemap.xml, robots.txt, and page-level structured metadata
- Configured and submitted Google Search Console property for sanghathi.com
- Applied SEO enhancements for Kethan's personal developer profile card

## I. Semester-Specific Product Logic

Problem: Sem-6 students had no 'Technical Work' tab, and mentor conversation navigation had an unauthorized route role mismatch breaking the mentor-mentee interaction flow.

- Added Semester 6 Technical Work tab with correct conditional rendering logic
- Fixed unauthorized route role mismatch in mentor conversation navigation
- Strengthened back-navigation actions and breadcrumb behavior across mentee pages

## J. Documentation & Project Reporting

- Authored 46 detailed worklog files covering every engineering action taken (docs/kethan/v1/)
- Created this master project report, contributor documentation, and ER diagram README
- Built summary.md as a living snapshot of metrics, impact, and archived logs
- Documented 91 unique files referenced across worklogs with backend/frontend classification

## 6.4 High-Impact Files Touched

| File Path | Worklog Refs |
|---|---|
| src/App.jsx | 3 |
| src/pages/DeveloperProfile.jsx | 2 |
| src/pages/AboutDevelopers.jsx | 2 |
| src/data/developers.js | 2 |
| src/hooks/useDraftPersistence.js | 2 |
| src/pages/Student/PrevAcademic.jsx | 2 |
| src/pages/Placement/InternshipDetails.jsx | 2 |
| src/pages/CareerReview/Mooc.jsx | 2 |

## 6.5 Worklog Archive

All engineering work is backed by 46 detailed worklog files archived at docs/kethan/v1/. Each log captures the problem identified, the fix applied, files modified, and test coverage status. Selected highlights:

- 2026-04-13: Security routing + MongoDB access hardening report
- 2026-04-13: 7-phase route hardening, logging, testing, and server script unification
- 2026-04-18: Cloudflare + Atlas backup runbooks and Bun runtime migration
- 2026-04-18: P0 message migration execution and one-to-one unique index enforcement
- 2026-04-18: Resend password reset migration and email template standardization
- 2026-04-18: Mentee fetch stability, skeleton loading, and responsive polish (Phases 1–3)
- 2026-04-18: HOD/Director route normalization and mentor avatar rollout
- 2026-04-18: Login page mobile responsive redesign + illustration parity
- 2026-04-19: IAT-6 CSV ingest execution, upload history restore, admin data guidance
- 2026-04-19: About Developers shortcards, footer contact inline, Kethan SEO upgrade
- 2026-04-19: Sem-6 Technical Work tab + mentor navigation role-mismatch fix

## 6.6 Contribution Philosophy

Kethan's contributions to Sanghathi prioritized production stability, security, and long-term maintainability over raw feature volume. Key principles applied throughout:

- Security-first: Every route, every role check, every access pattern was verified before moving on to UI work
- Documentation-driven: Every action was logged in a structured worklog before the next sprint began

- Phased rollouts: High-risk changes (schema migrations, message model cutover) were executed in controlled phases with rollback paths
- Real observability: Replaced scattered console logs with structured Winston logging so production issues surface cleanly
- End-to-end thinking: SEO, email deliverability, search console, and mobile responsiveness — not just backend APIs